

## PROJECT REPORT

# Multi-Agent AI Customer Support Assistant

*Using Retrieval-Augmented Generation (RAG) and Large Language Models*

Submitted as part of the Labmentix AI/ML Internship

Student: Anveiksha Sharma

*Fictional company used for knowledge base: TechMart Electronics*

Domain: Artificial Intelligence / Machine Learning

Category: Full-Stack Generative AI Application

July 2026

# Table of Contents

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>Table of Contents.....</b>                                     | <b>2</b>  |
| <b>Abstract.....</b>                                              | <b>5</b>  |
| <b>1. Introduction.....</b>                                       | <b>6</b>  |
| 1.1 Background.....                                               | 6         |
| 1.2 Motivation.....                                               | 6         |
| 1.3 Objectives.....                                               | 6         |
| 1.4 Scope of the Project.....                                     | 7         |
| <b>2. Problem Statement.....</b>                                  | <b>8</b>  |
| 2.1 Key Challenges Identified.....                                | 8         |
| <b>3. Existing Approaches and Related Concepts.....</b>           | <b>9</b>  |
| 3.1 Rule-Based / Decision-Tree Chatbots.....                      | 9         |
| 3.2 Single-Model LLM Chatbots.....                                | 9         |
| 3.3 Multi-Agent, RAG-Grounded Systems.....                        | 9         |
| <b>4. System Requirements.....</b>                                | <b>10</b> |
| 4.1 Functional Requirements.....                                  | 10        |
| 4.2 Non-Functional Requirements.....                              | 10        |
| 4.3 Hardware and Software Requirements.....                       | 10        |
| 4.3.1 Development Environment.....                                | 11        |
| 4.3.2 Production / Deployment Environment.....                    | 11        |
| <b>5. System Architecture.....</b>                                | <b>12</b> |
| 5.1 Architecture Flow.....                                        | 12        |
| 5.2 Component Responsibilities.....                               | 12        |
| <b>6. Technology Stack.....</b>                                   | <b>14</b> |
| 6.1 Rationale for Key Choices.....                                | 14        |
| <b>7. Detailed Module Design.....</b>                             | <b>15</b> |
| 7.1 Frontend Modules.....                                         | 15        |
| 7.2 Backend Entry Point — main.py.....                            | 15        |
| 7.3 Multi-Agent System.....                                       | 16        |
| 7.3.1 Shared Base Agent.....                                      | 16        |
| 7.3.2 Agent Router and Intent Detection.....                      | 16        |
| 7.3.3 The Five Specialized Agents.....                            | 17        |
| 7.3.4 Escalation Logic.....                                       | 17        |
| 7.4 Retrieval-Augmented Generation (RAG) Pipeline.....            | 17        |
| 7.4.1 Document Loading and Chunking — embeddings/embedder.py..... | 17        |
| 7.4.2 Embedding and Indexing — vectorstore/faiss_store.py.....    | 18        |
| 7.4.3 Retrieval — rag/retriever.py.....                           | 18        |
| 7.4.4 End-to-End RAG Workflow.....                                | 18        |
| 7.5 Database Layer.....                                           | 18        |
| 7.6 Request/Response Models (Pydantic Schemas).....               | 19        |

|                                                      |    |
|------------------------------------------------------|----|
| <b>8. API Reference</b>                              | 20 |
| <b>9. Agent Router — Worked Examples</b>             | 21 |
| Example 1 — Single agent (Billing)                   | 21 |
| Example 2 — Single agent (Technical)                 | 21 |
| Example 3 — Multi-agent (Billing + Technical)        | 21 |
| Example 4 — Complaint with escalation                | 21 |
| Example 5 — Default fallback (FAQ)                   | 21 |
| <b>10. Authentication and Security</b>               | 22 |
| 10.1 Password Handling                               | 22 |
| 10.2 Token Issuing and Verification                  | 22 |
| 10.3 Security Considerations and Limitations         | 22 |
| <b>11. Database Design</b>                           | 23 |
| <b>12. Frontend Design</b>                           | 24 |
| 12.1 Screens                                         | 24 |
| 12.2 State and Data Flow                             | 24 |
| <b>13. Testing and Evaluation</b>                    | 25 |
| 13.1 Test Cases                                      | 25 |
| 13.2 Evaluation Notes                                | 25 |
| <b>14. Deployment</b>                                | 26 |
| 14.1 Deployment Checklist                            | 26 |
| <b>15. Folder Structure</b>                          | 27 |
| <b>16. Challenges Faced and Solutions</b>            | 28 |
| <b>17. Learning Outcomes</b>                         | 29 |
| <b>18. Future Enhancements</b>                       | 30 |
| <b>19. Conclusion</b>                                | 31 |
| <b>20. Knowledge Base Content Overview</b>           | 32 |
| 20.1 FAQ.pdf                                         | 32 |
| 20.2 InstallationGuide.pdf                           | 32 |
| 20.3 Pricing.pdf                                     | 32 |
| 20.4 Products.pdf                                    | 32 |
| 20.5 RefundPolicy.pdf                                | 32 |
| 20.6 ShippingPolicy.pdf                              | 32 |
| 20.7 UserManual.pdf                                  | 32 |
| 20.8 Warranty.pdf                                    | 33 |
| <b>21. Sample Conversation Transcripts</b>           | 34 |
| Transcript 1 — Single-agent billing query            | 34 |
| Transcript 2 — Multi-agent billing + technical query | 34 |
| Transcript 3 — Escalated complaint                   | 34 |
| <b>22. Development Methodology and Timeline</b>      | 35 |
| <b>23. Risk Analysis</b>                             | 36 |
| <b>24. Glossary</b>                                  | 37 |

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>25. References.....</b>                             | <b>38</b> |
| <b>Appendix A: requirements.txt.....</b>               | <b>39</b> |
| <b>Appendix B: Environment Variables (.env).....</b>   | <b>40</b> |
| <b>Appendix C: Live Deployment Links.....</b>          | <b>41</b> |
| <b>Appendix D: Screenshots.....</b>                    | <b>42</b> |
| <b>Appendix E: Additional Code Listings.....</b>       | <b>49</b> |
| <b>E.1 backend/auth.py.....</b>                        | <b>49</b> |
| <b>E.2 backend/api/auth_routes.py.....</b>             | <b>49</b> |
| <b>E.3 backend/api/analytics_routes.py.....</b>        | <b>49</b> |
| <b>E.4 backend/llm.py.....</b>                         | <b>49</b> |
| <b>E.5 backend/models/user.py and feedback.py.....</b> | <b>50</b> |

## Abstract

Customer support teams increasingly rely on automation to manage the volume and diversity of incoming queries, but a single general-purpose chatbot often produces shallow or ungrounded answers when questions span billing, technical troubleshooting, product information, and complaint handling. This project presents the design and implementation of a Multi-Agent AI Customer Support Assistant that addresses this problem by combining three techniques: intent-based query routing, a team of specialized LLM-backed agents, and Retrieval-Augmented Generation (RAG) over a company knowledge base of PDF documents.

The system is built as a full-stack web application. A React (Vite) frontend provides a chat interface, authentication screens, and an analytics dashboard. A Python FastAPI backend exposes REST endpoints for authentication, chat, conversation history, and feedback. Incoming customer messages are first classified into one or more of six intent categories by an LLM-based classifier (with a keyword-based fallback for resilience), then routed to the corresponding specialized agent or agents — Billing, Technical Support, Product, Complaint, or FAQ. Each agent retrieves the most relevant chunks of company documentation from a FAISS vector index built over sentence-transformer embeddings, and combines that retrieved context with recent conversation history before generating a grounded response through a Llama 3 model served via the Groq API and orchestrated with LangChain.

Security is addressed through bcrypt password hashing and JWT-based session tokens, and all conversation turns are persisted in MongoDB so that history survives across sessions. The Complaint Agent additionally implements a lightweight escalation mechanism that flags conversations containing frustration-indicating language for human handoff. The completed system was deployed with the frontend on Vercel, the backend on Railway, and the database on MongoDB Atlas, and was manually tested across single-agent, multi-agent, and escalation scenarios. This report documents the problem motivation, requirements, architecture, detailed module design, API surface, database schema, security model, testing approach, deployment process, challenges encountered during development, and directions for future enhancement.

# 1. Introduction

## 1.1 Background

Conversational AI has moved through several generations of maturity: rule-based decision trees, intent-classification chatbots trained on narrow datasets, and now large language models (LLMs) capable of open-ended, fluent conversation. While LLMs are powerful general-purpose reasoners, they are not naturally grounded in any particular company's policies, product catalogue, or pricing — left unchecked, they can produce fluent but factually incorrect answers, a failure mode commonly known as hallucination. In a customer support setting, an incorrect answer about a refund window or a warranty period is not a cosmetic issue; it directly affects customer trust and can create financial or legal exposure for a business.

Retrieval-Augmented Generation (RAG) is the current standard architectural response to this problem: instead of relying purely on what the model memorized during training, the system retrieves relevant passages from a trusted, curated knowledge base at query time and supplies them to the model as context, so that the final answer is grounded in verifiable source material rather than the model's own possibly outdated or fabricated knowledge.

A second, related limitation of a single monolithic chatbot is domain breadth. Support queries in practice span very different domains — payments and invoices, technical troubleshooting, product specifications, and emotionally charged complaints — each of which benefits from a different tone, a different set of instructions, and in some cases a different escalation policy. A multi-agent architecture, in which a central router dispatches each query to one or more narrowly specialized agents, mirrors how a real support organization is structured into specialized teams, and tends to produce more consistent, appropriately-toned responses than a single prompt trying to do everything at once.

## 1.2 Motivation

The motivation for this project was to move beyond a toy chatbot demo and build something that reflects the architectural patterns actually used in production customer-support AI systems: intent detection, specialized agents, grounded retrieval, persistent session memory, and basic security — all wired together into a working, deployable full-stack application, rather than a single Python script or notebook. Building the full pipeline end-to-end, including authentication and a deployed live URL, was treated as an explicit goal so that the project could be evaluated as a realistic engineering artifact and not only as a proof-of-concept notebook.

## 1.3 Objectives

The specific objectives of the project are to:

- Design and implement an intent-detection layer that classifies an incoming customer query into one or more of six categories.
- Implement a central Agent Router that dispatches the query to the correct specialized agent(s) and aggregates their responses into a single coherent reply when more than one agent is triggered.
- Build five specialized agents (Billing, Technical Support, Product, Complaint, FAQ), each with a distinct system prompt, response style, and tone.
- Implement a Retrieval-Augmented Generation pipeline that chunks, embeds, indexes, and retrieves relevant passages from a knowledge base of company PDF documents.

- Persist users and conversation history in a database, and secure the application with hashed passwords and JWT-based session authentication.
- Implement an escalation mechanism so conversations showing signs of high customer dissatisfaction are automatically flagged for a human agent.
- Build a usable web frontend (login, registration, chat window, analytics dashboard) that consumes the backend API.
- Deploy the completed system to publicly reachable frontend and backend URLs backed by a managed cloud database.

## 1.4 Scope of the Project

The scope of this project covers a working end-to-end prototype suitable for demonstrating the architecture, evaluated against a fictional company ("TechMart Electronics") and its knowledge base documents (FAQ, installation guide, pricing, product catalogue, refund policy, shipping policy, user manual, and warranty). The scope explicitly excludes production-grade concerns that would be required for a real commercial deployment, such as multi-tenant support, horizontal auto-scaling, formal load testing, a human-agent handoff console, comprehensive automated test suites, and compliance certifications (e.g. PCI-DSS for real payment data). These are discussed as future enhancements in Section 18.

## 2. Problem Statement

Companies receive large volumes of customer queries daily, spanning very different domains: billing, technical support, product information, and complaints. A single monolithic chatbot often struggles to handle this diversity well — it must be prompted to behave correctly across every domain simultaneously, which in practice leads to generic, unfocused answers, inconsistent tone, and no clear way to apply domain-specific policies such as automatic escalation for a complaint versus a simple factual lookup for a product question.

This project addresses that by building a Multi-Agent AI System, where each agent specializes in one domain, and a central orchestrator (the Agent Router) decides which agent, or agents, should handle each incoming query. Grounding every agent's response in a retrieval step over real company documents further reduces the risk of the system inventing policy details, which is especially important for domains like billing (refund windows, charges) and warranty (coverage periods) where a wrong answer has direct real-world consequences.

### 2.1 Key Challenges Identified

| Challenge                          | Why it matters                                                                                          | Approach taken                                                                                   |
|------------------------------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Cross-domain queries               | A single query can touch more than one domain at once (e.g. a payment issue that also blocks a feature) | Router returns multiple matching intents; all matching agents run and their answers are combined |
| LLM hallucination on policy facts  | Wrong refund/warranty numbers damage trust and create liability                                         | RAG grounds every agent response in retrieved excerpts from real company PDF documents           |
| Inconsistent tone across domains   | A complaint needs empathy; a product question needs concise facts                                       | Each agent has an independent, tailored system prompt                                            |
| Escalation of unhappy customers    | Some issues genuinely require a human                                                                   | Complaint Agent scans for frustration-indicating phrases and flags the session                   |
| Session continuity                 | Customers expect the assistant to remember earlier turns in the conversation                            | Every message and response is persisted in MongoDB keyed by session ID                           |
| Availability if the LLM call fails | A support tool with a single point of failure is not acceptable                                         | Keyword-based fallback classifier keeps routing functional even without a live model response    |



## 3. Existing Approaches and Related Concepts

This section briefly situates the project's design choices against three broad categories of customer support automation that preceded and inform it.

### 3.1 Rule-Based / Decision-Tree Chatbots

The earliest generation of support bots used hand-written decision trees or keyword matching to route users through a fixed set of menu options. These systems are predictable and cheap to run, but brittle: any query phrased outside the anticipated set of keywords falls through to a generic "I didn't understand" response, and adding new coverage requires manually authoring new branches. The keyword fallback classifier in this project's Agent Router (Section 7.3.2) borrows this idea, but only as a safety net behind an LLM-based classifier, not as the primary mechanism.

### 3.2 Single-Model LLM Chatbots

A more recent and common pattern is to place a single LLM behind a chat widget with a broad system prompt covering all support scenarios. This is fast to build, but the prompt must simultaneously encode billing policy, technical troubleshooting steps, and complaint-handling tone, which tends to produce diluted behaviour in every domain compared with a prompt written for one purpose. It is also harder to attach domain-specific business logic (like automatic escalation rules) to a single undifferentiated prompt.

### 3.3 Multi-Agent, RAG-Grounded Systems

The architecture adopted here — a router in front of several narrowly-scoped agents, each grounded in retrieval over a curated knowledge base — reflects the pattern increasingly used in production AI support tooling, where specialization and retrieval grounding are treated as complementary techniques for improving both relevance and factual accuracy. It also has the practical benefit of being incrementally extensible: a new domain can be added as a new agent plus a new set of source documents, without having to rewrite an existing monolithic prompt.

## 4. System Requirements

### 4.1 Functional Requirements

| ID    | Requirement                                                                                                                  |
|-------|------------------------------------------------------------------------------------------------------------------------------|
| FR-1  | The system shall allow a new user to register with name, email, and password.                                                |
| FR-2  | The system shall allow a registered user to log in and receive a signed session token.                                       |
| FR-3  | The system shall classify each incoming chat message into one or more of six intent categories.                              |
| FR-4  | The system shall route each message to the specialized agent(s) matching its detected intent(s).                             |
| FR-5  | The system shall retrieve relevant passages from the company knowledge base for every agent response.                        |
| FR-6  | The system shall combine multiple agents' answers into a single response when a query spans more than one domain.            |
| FR-7  | The system shall persist every user and assistant message with a session identifier and timestamp.                           |
| FR-8  | The system shall return prior conversation history for a given session on request.                                           |
| FR-9  | The system shall automatically flag a conversation for human escalation when complaint language crosses a defined threshold. |
| FR-10 | The system shall reject requests to protected endpoints that do not carry a valid session token.                             |
| FR-11 | The system shall allow a user to submit a thumbs up/down feedback rating for a given agent response.                         |
| FR-12 | The system shall expose aggregate analytics (conversation counts, per-agent usage) via an API endpoint.                      |

### 4.2 Non-Functional Requirements

| ID    | Requirement      | Notes                                                                                                 |
|-------|------------------|-------------------------------------------------------------------------------------------------------|
| NFR-1 | Usability        | Chat interface should resemble a familiar messaging UI with visible typing indicator                  |
| NFR-2 | Security         | Passwords never stored in plain text; all protected routes require a valid signed token               |
| NFR-3 | Maintainability  | New agents can be added by subclassing a shared BaseAgent without touching routing logic              |
| NFR-4 | Resilience       | A missing or failing LLM call must not crash the request; fallback logic must apply                   |
| NFR-5 | Portability      | Backend and frontend must run locally with only a Groq key and a MongoDB URI as external dependencies |
| NFR-6 | Response latency | Typical single-agent response should complete within a few seconds under normal API conditions        |

### 4.3 Hardware and Software Requirements

### 4.3.1 Development Environment

- OS: Windows, macOS, or Linux
- Python 3.10+ (project tested with the standard CPython interpreter; requirements.txt has no GPU dependency)
- Node.js 18+ and npm for the Vite/React frontend
- A free Groq API key (console.groq.com) for LLM inference
- A free-tier MongoDB Atlas cluster (M0) for the database
- Minimum 4 GB RAM recommended (the sentence-transformer embedding model runs on CPU)

### 4.3.2 Production / Deployment Environment

- Frontend hosting: Vercel (static build of the Vite app)
- Backend hosting: Railway (Python/uvicorn process)
- Database: MongoDB Atlas free M0 cluster
- Environment variables: GROQ\_API\_KEY, MONGODB\_URI, MONGODB\_DB\_NAME, JWT\_SECRET

## 5. System Architecture

The system follows a layered architecture: the customer interacts with a React-based web chat interface, which communicates with a FastAPI backend over REST APIs. The backend first checks the user's login session, then passes the query through an Intent Detection step, which decides which of six categories the query belongs to. The Agent Router then dispatches the query to one or more of five specialized agents. Each agent retrieves relevant context from a FAISS vector store (built over company PDF documents using RAG), combines that context with the conversation history, and generates a response via an LLM (Llama 3, accessed through Groq and LangChain). Every message and response is stored in MongoDB, tagged with a session ID and timestamp, so conversation history persists across visits.

### 5.1 Architecture Flow

```

Customer
-> Web Chat Interface (React)
-> Backend API (FastAPI)
-> Auth Check (JWT)
-> Intent Detection (LLM classifier, keyword fallback)
-> Agent Router
    -> Billing Agent      -\
    -> Technical Agent   |
    -> Product Agent     |-- RAG Retrieval (FAISS + Embeddings)
    -> Complaint Agent   |      -> Company PDF Documents
    -> FAQ Agent         -/
-> LLM Response Generation (LangChain + Groq / Llama 3)
-> Response Aggregator
-> Final Response -> Customer
-> Conversation Memory (MongoDB)
  
```

### 5.2 Component Responsibilities

| Component                                  | Responsibility                                                                                         |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------|
| React Frontend                             | Login/registration screens, chat window with message history and typing indicator, analytics dashboard |
| FastAPI Backend (main.py)                  | Exposes /health, /chat, /history/{session_id}, mounts auth and analytics routers                       |
| Auth Module (auth.py)                      | Password hashing, JWT issuing and verification, current-user dependency for protected routes           |
| Agent Router (agents/router.py)            | Intent detection and dispatch to one or more specialized agents, response aggregation                  |
| Specialized Agents (agents/*.py)           | Domain-specific system prompts and response behaviour, built on a shared BaseAgent                     |
| RAG Retriever (rag/retriever.py)           | Bridges the embedding module and the FAISS store; exposes retrieve(query)                              |
| Embeddings Module (embeddings/embedder.py) | Loads and chunks PDF documents, generates embeddings for chunks and queries                            |
| Vector Store (vectorstore/faiss_store.py)  | In-memory FAISS index construction and cosine-similarity search                                        |
| Database Module (database.py)              | MongoDB connection, user CRUD, message persistence, feedback, analytics                                |

| Component            | Responsibility                                                                         |
|----------------------|----------------------------------------------------------------------------------------|
|                      | aggregation                                                                            |
| LLM Wrapper (llm.py) | Thin LangChain + Groq wrapper exposing a single chat(system_prompt, user_message) call |

## 6. Technology Stack

| Layer           | Technology Used                                                              |
|-----------------|------------------------------------------------------------------------------|
| Frontend        | React (Vite), Tailwind CSS, Axios                                            |
| Backend         | Python, FastAPI                                                              |
| Authentication  | JWT sessions, bcrypt password hashing                                        |
| AI Model        | Llama 3 (llama-3.1-8b-instant), via Groq API, orchestrated through LangChain |
| Embeddings      | sentence-transformers (all-MiniLM-L6-v2)                                     |
| Vector Database | FAISS (IndexFlatIP, cosine similarity over normalized vectors)               |
| Database        | MongoDB (users and conversation history)                                     |
| PDF Parsing     | pypdf                                                                        |

### 6.1 Rationale for Key Choices

- Groq was chosen as the LLM inference provider because it offers very low-latency inference for open models such as Llama 3, which keeps chat response times acceptable for an interactive support tool without requiring a locally-hosted GPU.
- LangChain is used as a thin orchestration layer around the Groq client so that prompt templates are declared once and reused consistently by every agent, rather than each agent formatting a raw string prompt independently.
- sentence-transformers' all-MiniLM-L6-v2 was chosen for embeddings because it runs entirely on CPU, is small enough to load quickly at server startup, and is widely used as a solid general-purpose baseline for semantic similarity tasks.
- FAISS was chosen over a managed/cloud vector database because the knowledge base here is small (eight PDF documents), so an in-memory flat index rebuilt at process startup is simpler to operate than a hosted vector database, while still being a realistic stand-in for how a larger system would use the same retrieval interface.
- MongoDB's flexible document model suits the variable shape of a chat message (which may or may not carry an agent tag, escalation flag, or feedback rating) better than a fixed relational schema would.
- JWT was chosen over server-side sessions so that the stateless FastAPI backend does not need a shared session store, which simplifies horizontal deployment on platforms like Railway.

## 7. Detailed Module Design

### 7.1 Frontend Modules

The frontend is a Vite-built React application organized into pages, reusable components, hooks, and a services layer that wraps API calls.

| File                                  | Responsibility                                                                  |
|---------------------------------------|---------------------------------------------------------------------------------|
| App.jsx                               | Top-level routing between login, registration, chat, and analytics pages        |
| pages/LoginPage.jsx                   | Email/password login form, calls the auth service, stores the returned token    |
| pages/RegisterPage.jsx                | Name/email/password registration form                                           |
| pages/SupportChatPage.jsx             | Hosts the ChatWindow and ChatInput components for an active session             |
| pages/AnalyticsPage.jsx               | Displays conversation counts and per-agent usage returned by /analytics         |
| components/ChatWindow.jsx             | Renders the scrolling list of MessageBubble components and the typing indicator |
| components/MessageBubble.jsx          | Renders a single user or assistant message, styled by sender                    |
| components/ChatInput.jsx              | Text input and send button; submits the message to the chat hook                |
| hooks/useAuth.js                      | Wraps login/registration calls and manages the stored auth token                |
| hooks/useChat.js                      | Manages sending messages to /chat and updating local message state              |
| services/api.js                       | Axios instance pre-configured to attach the Authorization header automatically  |
| services/auth.js                      | Thin wrapper functions for the /auth/login and /auth/register endpoints         |
| styles/index.css / tailwind.config.js | Tailwind CSS setup and global styling                                           |

### 7.2 Backend Entry Point — main.py

main.py wires together the FastAPI application: it enables permissive CORS for development, mounts the authentication and analytics routers, and defines the /health, /chat, and /history/{session\_id} endpoints. The /chat endpoint is the central request path — it resolves or creates a session ID, loads recent history from MongoDB for context, persists the incoming user message, calls route\_query() from the Agent Router, persists the assistant's response tagged with the agent(s) used, and returns a structured ChatResponse containing the answer, which agents were used, which intents were detected, which knowledge-base sources were cited, and whether the conversation was escalated.

```
@app.post("/chat", response_model=ChatResponse)
def chat(req: ChatRequest, current_user: dict = Depends(get_current_user)):
    session_id = req.session_id or str(uuid.uuid4())
    past = database.get_history(session_id, limit=10)
    history_text = "\n".join([f"{m['role']}: {m['content']}" for m in past]) if past
    else ""
    database.save_message(session_id, "user", req.message)
    result = route_query(req.message, history_text)
    database.save_message(session_id, "assistant", result["answer"],
```

```

        agent=",".join(result["agents_used"]))
    return ChatResponse(session_id=session_id, answer=result["answer"],
                        agents_used=result["agents_used"], intents=result["intents"],
                        sources=result["sources"], escalated=result["escalated"])

```

Every protected route (/chat, /history/{session\_id}, /feedback, /analytics) declares `current_user: dict = Depends(get_current_user)`, so FastAPI's dependency-injection system automatically rejects unauthenticated requests before the route body ever runs.

## 7.3 Multi-Agent System

### 7.3.1 Shared Base Agent

All five specialized agents inherit from a common `BaseAgent` class, which implements the shared retrieve -> prompt -> generate flow. Only the name and `system_prompt` class attributes differ between agents; the retrieval and prompt-assembly logic is written once.

```

class BaseAgent:
    name = "Base Agent"
    system_prompt = "You are a helpful customer support assistant for TechMart Electronics."

    def handle(self, query: str, history_text: str = "") -> dict:
        retrieved = retriever.retrieve(query, top_k=3)
        context_text = "\n\n".join([f"[{r['source']}]: {r['text']}" for r in
retrieved])
        full_prompt = f"""Conversation so far:
{history_text or '(no previous messages)'}

Relevant company documents:
{context_text or '(no matching documents found)'}

Customer query:
{query}

Answer the customer clearly and concisely using the documents above where relevant.
If the documents don't cover the question, say so honestly and offer to escalate to a
human agent."""
        answer = llm.chat(self.system_prompt, full_prompt)
        return {"agent": self.name, "answer": answer,
                "sources": list({r["source"] for r in retrieved})}

```

This design keeps each concrete agent file extremely small — typically just a name and a tailored system prompt — which makes the system easy to extend with a new domain in the future (see Section 18).

### 7.3.2 Agent Router and Intent Detection

The router first calls `detect_intent()`, which sends the query to the LLM with a strict classification prompt asking it to return a comma-separated list of matching category names from {billing, technical, product, complaint, faq}. If the LLM call fails for any reason (missing API key, rate limit, network error — signalled by the wrapper returning a string beginning with "[LLM]"), the router falls back to a keyword-matching classifier so that the system degrades gracefully rather than failing outright.

```

def detect_intent(query: str):
    raw = llm.chat(CLASSIFIER_PROMPT, query, temperature=0.0)

```



```

if raw.startswith("[LLM]"):
    return _keyword_fallback(query)
categories = [c.strip().lower() for c in raw.replace("\n", " ").split(",")]
valid = [c for c in categories if c in INTENT_CATEGORIES]
return valid or _keyword_fallback(query)

def route_query(query: str, history_text: str = ""):
    intents = detect_intent(query)
    results = [AGENTS[intent].handle(query, history_text) for intent in intents]
    if len(results) == 1:
        final_answer = results[0]["answer"]
    else:
        combined = "\n\n".join([f"({r['agent']}): {r['answer']}" for r in results])
        final_answer = combined
    return {"intents": intents, "agents_used": [r["agent"] for r in results],
            "answer": final_answer,
            "sources": sorted({s for r in results for s in r["sources"]}),
            "escalated": any(r.get("escalated") for r in results)}

```

### 7.3.3 The Five Specialized Agents

| Agent                   | Domain focus                                              | Notable behaviour                                                                                                                 |
|-------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Billing Agent           | Payments, subscriptions, invoices, refunds                | Instructed to be precise about policy numbers (days, percentages); replies capped near 120 words                                  |
| Technical Support Agent | Login issues, password resets, installation, errors, bugs | Instructed to give clear step-by-step troubleshooting instructions                                                                |
| Product Agent           | Features, pricing, comparisons, availability              | Instructed to be specific and use the retrieved documents directly                                                                |
| Complaint Agent         | Dissatisfied customers, service failures                  | Overrides handle() to additionally run an escalation check against a keyword list; appends a human hand-off notice when triggered |
| FAQ Agent               | General policy, account help, contact information         | Shorter response cap (under 100 words), friendlier tone, acts as the fallback category when intent is unclear                     |

### 7.3.4 Escalation Logic

The Complaint Agent maintains a list of escalation-trigger phrases (for example references to legal action, fraud, repeated unresolved issues, or explicit account cancellation intent). If any of these phrases appear in the customer's message, the agent sets `escalated: true` on its result and appends a notice to the answer informing the customer that a human specialist will follow up within 24 hours. Because `route_query()` computes `escalated` as `any(r.get('escalated') for r in results)`, a multi-agent response is marked escalated as soon as any one of the involved agents flags it, even if the Complaint Agent was only one of several agents triggered by the query.

## 7.4 Retrieval-Augmented Generation (RAG) Pipeline

### 7.4.1 Document Loading and Chunking — `embeddings/embedder.py`

At startup, every PDF in the `knowledge_base/` folder is read with `pypdf` and its extracted text is split using a simple fixed-size sliding-window chunker: 400-character chunks with a 50-character overlap between consecutive chunks. The overlap ensures that a sentence or fact that happens to fall across a chunk boundary is still likely to appear intact in at least one chunk.

```
CHUNK_SIZE = 400
CHUNK_OVERLAP = 50

def _chunk_text(text: str, chunk_size: int = CHUNK_SIZE, overlap: int = CHUNK_OVERLAP):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end].strip())
        start += chunk_size - overlap
    return [c for c in chunks if c]
```

#### 7.4.2 Embedding and Indexing — `vectorstore/faiss_store.py`

Each chunk is embedded using the `all-MiniLM-L6-v2` sentence-transformer model, with embeddings normalized so that a FAISS `IndexFlatIP` (inner product) index is mathematically equivalent to cosine-similarity search. The `FaissStore` class wraps index construction and top-k search, returning each match's original text, its source PDF filename, and its similarity score.

#### 7.4.3 Retrieval — `rag/retriever.py`

The `Retriever` class ties the embedding module and the FAISS store together behind a single `retrieve(query, top_k=3)` call. The index is built exactly once, when the FastAPI process starts (`retriever = Retriever()` at import time), so subsequent chat requests only pay the cost of embedding the incoming query and performing a similarity search, not re-indexing the documents.

#### 7.4.4 End-to-End RAG Workflow

1. Company documents (PDFs) are loaded from the `knowledge_base/` folder.
2. Each document is split into overlapping text chunks (~400 characters, 50 character overlap).
3. Each chunk is converted into a vector embedding using sentence-transformers (`all-MiniLM-L6-v2`).
4. Embeddings are stored in a FAISS index for fast similarity search.
5. When a customer asks a question, the query is embedded the same way.
6. FAISS retrieves the top-matching chunks by cosine similarity.
7. Retrieved chunks are passed to the LLM as context, along with the conversation history.
8. The LLM generates a final answer grounded in the retrieved company documents.

### 7.5 Database Layer

The database module wraps a MongoDB connection and exposes small, purpose-specific functions rather than a general-purpose ORM: `get_user_by_email`, `create_user`, `save_message`, `get_history`, `save_feedback`, and `get_analytics`. Every chat turn is stored as an individual document with a role ("user" or "assistant"), the message content, an optional agent tag (comma-separated list of agent names for assistant messages), and a timestamp,

keyed by a session\_id string so a full conversation can be reconstructed with a single query filtered and sorted by that field.

## 7.6 Request/Response Models (Pydantic Schemas)

| Model           | Fields                                                       | Used by             |
|-----------------|--------------------------------------------------------------|---------------------|
| UserRegister    | name, email, password                                        | POST /auth/register |
| UserLogin       | email, password                                              | POST /auth/login    |
| TokenResponse   | access_token, token_type, name, email                        | Both auth endpoints |
| ChatRequest     | session_id (optional), message                               | POST /chat          |
| ChatResponse    | session_id, answer, agents_used, intents, sources, escalated | POST /chat          |
| FeedbackRequest | session_id, agent, rating ("up"/"down")                      | POST /feedback      |

## 8. API Reference

| Method | Endpoint              | Auth required | Description                                                          |
|--------|-----------------------|---------------|----------------------------------------------------------------------|
| GET    | /health               | No            | Basic liveness check, returns {"status": "ok"}                       |
| POST   | /auth/register        | No            | Creates a new user; returns a signed access token                    |
| POST   | /auth/login           | No            | Verifies credentials; returns a signed access token                  |
| POST   | /chat                 | Yes           | Submits a customer message; returns the routed, RAG-grounded answer  |
| GET    | /history/{session_id} | Yes           | Returns all persisted messages for a given session                   |
| POST   | /feedback             | Yes           | Records a thumbs up/down rating against a session and agent          |
| GET    | /analytics            | Yes           | Returns aggregate conversation counts and per-agent usage statistics |

All protected endpoints expect an Authorization: Bearer <token> header, where <token> is the JWT returned by /auth/login or /auth/register. A missing or invalid token results in an HTTP 401 response before the endpoint's own logic executes, since the check is implemented as a shared FastAPI dependency (get\_current\_user).

## 9. Agent Router — Worked Examples

The Agent Router can dispatch a single query to multiple agents when it spans more than one domain. The following worked examples illustrate single-agent, multi-agent, and escalation routing.

### Example 1 — Single agent (Billing)

Query: "What's your refund policy?"

Detected intent: billing. Only the Billing Agent runs; it retrieves the refund-policy passages from RefundPolicy.pdf and answers with the exact refund window quoted from the document.

### Example 2 — Single agent (Technical)

Query: "My app keeps crashing on login."

Detected intent: technical. The Technical Support Agent retrieves relevant troubleshooting passages and returns step-by-step instructions.

### Example 3 — Multi-agent (Billing + Technical)

Query: "I paid yesterday but Premium is still locked." This is classified as touching both Billing (a payment was made) and Technical Support (a feature is not unlocking as expected), so both the Billing Agent and the Technical Support Agent generate a response, which are then combined into a single reply shown to the customer.

### Example 4 — Complaint with escalation

Query: "This is the third time I've had this issue, I want a refund now!" This routes to both Complaint and Billing. The Complaint Agent detects the escalation-trigger phrase "third time" and flags the conversation; the final response is marked escalated: true and includes a note that a human specialist will follow up within 24 hours.

### Example 5 — Default fallback (FAQ)

Query: "What are your support hours?" No domain-specific keywords match strongly, so the query is classified as faq, and the FAQ Agent answers concisely from the general company documentation.

## 10. Authentication and Security

User passwords are hashed with bcrypt before being stored in MongoDB; plain-text passwords are never stored. On successful login or registration, the backend issues a signed JWT (JSON Web Token) valid for 24 hours. The frontend stores this token and attaches it automatically as an Authorization header on every request. The /chat and /history API endpoints are protected, so only authenticated users can access them.

### 10.1 Password Handling

```
def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")

def verify_password(password: str, hashed: str) -> bool:
    return bcrypt.checkpw(password.encode("utf-8"), hashed.encode("utf-8"))
```

### 10.2 Token Issuing and Verification

create\_access\_token() embeds the user's email as the sub claim along with a 24-hour expiry, and signs the token with a server-side JWT\_SECRET using the HS256 algorithm. get\_current\_user() is used as a FastAPI dependency on every protected route: it decodes and verifies the token, raising an HTTP 401 error on any failure (invalid signature, expired token, or malformed payload), and otherwise returns the identified user's email for the route to use.

### 10.3 Security Considerations and Limitations

- JWT\_SECRET defaults to a placeholder development value if not set via environment variable; production deployments must set a strong random secret.
- Tokens are not currently revocable before their 24-hour expiry (no server-side blacklist), which is an accepted limitation for this prototype scope.
- CORS is configured permissively (allow\_origins=["\*"]) for development convenience; a production deployment should restrict this to the known frontend origin.
- The current design stores the JWT client-side; a future iteration could consider httpOnly cookies to reduce exposure to cross-site scripting.

## 11. Database Design

MongoDB was chosen for its flexible document schema, which suits chat messages whose optional fields (agent tag, escalation flag) vary by message type. The database is organized into the following logical collections.

| Collection | Key fields                                                   | Purpose                                                           |
|------------|--------------------------------------------------------------|-------------------------------------------------------------------|
| users      | name, email (unique), password (bcrypt hash), created_at     | Stores registered user accounts                                   |
| messages   | session_id, role (user/assistant), content, agent, timestamp | Stores every turn of every conversation for history and analytics |
| feedback   | session_id, agent, rating (up/down), timestamp               | Stores per-response feedback ratings used for analytics           |

`get_history(session_id, limit)` queries the messages collection filtered by `session_id`, sorted by timestamp, and limited to the most recent N turns to keep the context passed back to the LLM bounded in size. `get_analytics()` aggregates conversation counts and per-agent usage counts across the messages collection to power the AnalyticsPage on the frontend.

## 12. Frontend Design

The frontend is a single-page application built with React and Vite, styled with Tailwind CSS utility classes, and communicating with the backend exclusively through the Axios-based services/api.js client, which centrally attaches the Authorization header once a user is logged in.

### 12.1 Screens

- Login page — email/password form; on success stores the returned token and redirects to the chat screen.
- Registration page — name/email/password form; behaves the same as login on success, since registration also returns a usable access token immediately.
- Support Chat page — the main interface: a scrolling message list (ChatWindow), individual message bubbles styled differently for the user versus the assistant, a typing indicator while waiting for a response, and a text input with a send button (ChatInput).
- Analytics page — a simple dashboard reading from GET /analytics, showing total conversation counts and how often each specialized agent was invoked.

### 12.2 State and Data Flow

The useAuth hook centralizes login/registration calls and token storage, while the useChat hook manages the message list for the active session and calls POST /chat whenever the user sends a new message, appending both the user's message and the returned assistant answer to local state so the chat window updates immediately.



## 13. Testing and Evaluation

The system was tested manually with a range of queries across all five agent domains, including queries that should route to a single agent (e.g. refund policy questions) and queries that should route to multiple agents at once (e.g. a combined billing and technical issue). Escalation logic in the Complaint Agent was tested with frustration-indicating phrases to confirm it correctly flags conversations for human handoff. Retrieval quality was checked by confirming that responses cited real, accurate policy details (e.g. exact day counts for refunds and warranty periods) rather than hallucinated information.

### 13.1 Test Cases

| #  | Test input                                                         | Expected routing                        | Result                                                               |
|----|--------------------------------------------------------------------|-----------------------------------------|----------------------------------------------------------------------|
| 1  | "What's your refund policy?"                                       | Billing only                            | Pass — correct refund window returned, sourced from RefundPolicy.pdf |
| 2  | "My app keeps crashing on login"                                   | Technical only                          | Pass — step-by-step troubleshooting returned                         |
| 3  | "I paid yesterday but Premium is still locked"                     | Billing + Technical                     | Pass — combined response from both agents                            |
| 4  | "This is the third time I've had this issue, I want a refund now!" | Complaint + Billing, escalated=true     | Pass — escalation notice appended                                    |
| 5  | "What are your support hours?"                                     | FAQ (default fallback)                  | Pass                                                                 |
| 6  | Chat request with no Authorization header                          | HTTP 401                                | Pass — request rejected before reaching handler                      |
| 7  | Login with wrong password                                          | HTTP 401 "Invalid email or password"    | Pass                                                                 |
| 8  | Register with an already-used email                                | HTTP 400 "Email already registered"     | Pass                                                                 |
| 9  | GET /history/{session_id} for a multi-turn session                 | All prior turns returned in order       | Pass                                                                 |
| 10 | LLM call intentionally disabled (no GROQ_API_KEY)                  | Router falls back to keyword classifier | Pass — system remains functional in degraded mode                    |

### 13.2 Evaluation Notes

Because this is a prototype built around a fictional company's documents, evaluation focused on structural correctness (did the right agent(s) fire, was the escalation flag set correctly, did retrieved sources match the expected PDF) rather than a large-scale quantitative benchmark. A natural next step, discussed in Section 18, is a more formal evaluation harness with a labelled set of test queries and automatic scoring of routing accuracy and retrieval precision.

## 14. Deployment

The application is deployed and publicly accessible:

| Component | Platform      | Notes                                                                             |
|-----------|---------------|-----------------------------------------------------------------------------------|
| Frontend  | Vercel        | Static build of the Vite app; VITE_API_URL points to the Railway backend          |
| Backend   | Railway       | Runs uvicorn backend.main:app; environment variables set in the Railway dashboard |
| Database  | MongoDB Atlas | Free M0 cluster; network access opened for the backend host                       |

### 14.1 Deployment Checklist

- Push the project to a GitHub repository (both Vercel and the backend host deploy from GitHub).
- Backend: set the build command to `pip install -r requirements.txt` and the start command to `uvicorn backend.main:app --host 0.0.0.0 --port $PORT`, and set `GROQ_API_KEY`, `MONGODB_URI`, `MONGODB_DB_NAME`, and `JWT_SECRET` as environment variables.
- Frontend: set the project root directory to `frontend/`, and set `VITE_API_URL` to the deployed backend URL.
- MongoDB Atlas: open Network Access to allow connections from the backend host (`0.0.0.0/0` for a simple setup, or a specific IP range for tighter security).

## 15. Folder Structure

```

customer-support-ai/
├── frontend/
│   ├── src/
│   │   ├── components/    (ChatInput, ChatWindow, MessageBubble)
│   │   ├── pages/         (LoginPage, RegisterPage, SupportChatPage, AnalyticsPage)
│   │   ├── hooks/         (useAuth, useChat)
│   │   ├── services/      (api.js, auth.js)
│   │   └── styles/        (index.css)
│   ├── package.json, vite.config.js, tailwind.config.js
│   └── index.html
├── backend/
│   ├── api/               (auth_routes.py, analytics_routes.py)
│   ├── agents/            (billing, technical, product, complaint, faq, router,
base_agent)
│   ├── embeddings/        (embedder.py – PDF loading, chunking, embedding generation)
│   ├── vectorstore/       (faiss_store.py – FAISS index build and search)
│   ├── rag/               (retriever.py – ties embeddings + vectorstore together)
│   ├── models/            (user.py, feedback.py – Pydantic schemas)
│   ├── database/          (MongoDB connection and queries)
│   ├── auth.py, llm.py, main.py
├── knowledge_base/        (8 company PDF documents)
├── datasets/
├── requirements.txt
└── README.md

```

## 16. Challenges Faced and Solutions

| Challenge                                                               | Solution adopted                                                                                                                                           |
|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Classifying queries that span more than one support domain              | The router treats intent detection as a multi-label task (comma-separated categories) rather than a single-label choice, and runs every matching agent     |
| Keeping the system usable if the LLM API is unavailable                 | A deterministic keyword-matching fallback classifier is triggered whenever the LLM wrapper reports a failure                                               |
| Avoiding hallucinated policy details (refund windows, warranty periods) | Every agent response is grounded in top-k retrieved chunks from the actual company PDFs before generation                                                  |
| Deciding when a complaint needs a human                                 | A curated list of escalation-trigger phrases is checked against the raw query text in the Complaint Agent                                                  |
| Keeping conversation context bounded                                    | <code>get_history()</code> is called with a limit (10 most recent turns) so the prompt passed to the LLM does not grow unbounded over a long session       |
| Coordinating a React frontend and a separately-deployed FastAPI backend | A single centralized Axios client ( <code>services/api.js</code> ) with an environment-configurable base URL and automatic Authorization header attachment |

## 17. Learning Outcomes

Building this project provided hands-on experience across the full stack required for a modern generative-AI application:

- Designing prompts for a classification task (intent detection) separately from prompts for a generation task (agent answers), and understanding why the two require different temperature settings and different levels of prompt strictness.
- Implementing Retrieval-Augmented Generation from first principles — chunking, embedding, indexing, and retrieval — rather than only using a pre-built RAG framework, which clarified exactly what each stage of the pipeline contributes.
- Practical experience with FastAPI's dependency-injection system for authentication, and with JWT-based stateless session management.
- Structuring a multi-agent system so that shared logic (the retrieve-then-generate flow) lives in one base class, while each specialized agent stays a small, easily reviewed file.
- End-to-end deployment experience: separating a frontend static build (Vercel), a backend API process (Railway), and a managed database (MongoDB Atlas), and wiring environment variables correctly across all three.
- Appreciating the importance of graceful degradation (the keyword fallback classifier) in an AI system that depends on a third-party inference API.

## 18. Future Enhancements

- Replace the fixed-size sliding-window chunker with a semantic or structure-aware chunker (e.g. splitting on headings/sections) to improve retrieval precision on longer documents.
- Add a human-agent handoff console so escalated conversations can be picked up and answered by a real support agent within the same interface.
- Persist and expose retrieval confidence scores to the frontend so low-confidence answers can be visually flagged to the customer.
- Add automated regression tests (e.g. pytest test suite covering routing, escalation, and auth) to replace the current manual testing approach.
- Introduce a formal evaluation harness with a labelled set of test queries to quantitatively measure routing accuracy and RAG retrieval precision/recall over time.
- Support streaming responses (token-by-token) from the LLM to reduce perceived latency in the chat interface.
- Add multi-language support for both the knowledge base and the chat interface.
- Introduce rate limiting and token revocation (a server-side denylist or short-lived refresh tokens) to harden the authentication system for production use.
- Extend the Analytics dashboard with time-series views and customer satisfaction trends derived from the feedback collection.

## 19. Conclusion

This project demonstrates a practical, working implementation of a multi-agent AI customer support system, combining intent detection, agent orchestration, retrieval-augmented generation, and secure session management into a single full-stack application. It reflects the core architecture patterns used in production AI customer support systems today: specialized agents coordinated by a central router, grounded in a real knowledge base rather than relying purely on an LLM's general knowledge. The completed system was deployed to publicly reachable URLs and tested across single-agent, multi-agent, and escalation scenarios, providing a solid foundation that can be extended in the directions outlined in Section 18.

## 20. Knowledge Base Content Overview

The RAG pipeline is grounded in eight short company documents stored as PDFs in the `knowledge_base/` folder, written for the fictional company "TechMart Electronics" used throughout this project. Each document maps naturally onto one or more of the specialized agents' domains. A brief summary of each document's content follows, so the retrieval and agent-response examples in earlier sections can be cross-checked against their real source material.

### 20.1 FAQ.pdf

Covers general questions such as support hours (Monday to Saturday, 9 AM to 9 PM IST, reachable via chat, email, or phone), how to track an order from the account menu, and free installation availability for large appliances within 7 days of delivery. Primarily used by the FAQ Agent.

### 20.2 InstallationGuide.pdf

Describes installation procedures for large appliances (automatic technician scheduling within 24-48 hours), television setup steps, and a note that air conditioners require professional installation only. Used mainly by the Technical Support Agent and the Product Agent.

### 20.3 Pricing.pdf

Describes the Premium Membership (Rs. 499/year, free express delivery, early sale access, 5% extra cashback), EMI availability on purchases above Rs. 3000 for 3 to 24 months, tax-inclusive pricing, seasonal sale discounts up to 70%, and price-drop email alerts. Used mainly by the Billing Agent and the Product Agent.

### 20.4 Products.pdf

Provides a catalogue overview across smartphones (budget to flagship, minimum 1-year warranty), laptops (student to gaming configurations), and televisions (LED/QLED/OLED, 32 to 75 inches). Used mainly by the Product Agent.

### 20.5 RefundPolicy.pdf

States the core billing policy figures used throughout this report's examples: returns accepted within 10 days of delivery for unused items in original packaging, refunds processed within 5-7 business days after quality inspection, refunds credited to the original payment method (or bank transfer for Cash on Delivery), free replacement for defective items within 15 days, non-refundable digital products and gift cards, and automatic reversal of payments deducted against failed orders. This document is the primary source for the Billing Agent.

### 20.6 ShippingPolicy.pdf

Describes standard delivery (3-5 business days), express delivery (1-2 business days in select cities), free shipping above Rs. 999, extended delivery windows for large appliances (5-8 business days), and SMS/email tracking notifications at every stage. Used mainly by the Billing and FAQ Agents.

### 20.7 UserManual.pdf



Walks through account creation (sign-up with email OTP verification), placing an order (browse, select variant, checkout), and tracking an order's real-time status from the account menu. Used mainly by the FAQ and Technical Support Agents.

### **20.8 Warranty.pdf**

States warranty durations by category: smartphones and laptops carry a 1-year standard warranty, large appliances carry 1-2 years with an optional extension up to 5 years, and accessories carry a 6-month warranty; explicitly excludes physical damage, liquid damage, and unauthorized repairs from coverage. This is the primary source for warranty-related Billing and Technical Support Agent responses.

## 21. Sample Conversation Transcripts

The following illustrative transcripts show the expected shape of a session end-to-end, including which agent(s) responded and which knowledge-base documents were cited as sources.

### Transcript 1 — Single-agent billing query

| Turn | Speaker                   | Content                                                                                                                                                                      |
|------|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Customer                  | "How long do I have to return a product if I don't like it?"                                                                                                                 |
| 2    | Assistant (Billing Agent) | Explains the 10-day return window for unused items in original packaging, and that refunds are processed within 5-7 business days after inspection. Source: RefundPolicy.pdf |

### Transcript 2 — Multi-agent billing + technical query

| Turn | Speaker                         | Content                                                                                                                                                                                                                                                |
|------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Customer                        | "I paid yesterday but Premium is still locked."                                                                                                                                                                                                        |
| 2    | Assistant (Billing + Technical) | Billing Agent confirms Premium membership pricing/activation expectations from Pricing.pdf; Technical Agent offers troubleshooting steps (re-login, check account status page, contact support if unresolved after 24 hours). Combined into one reply. |

### Transcript 3 — Escalated complaint

| Turn | Speaker                         | Content                                                                                                                                                                                                                                                    |
|------|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Customer                        | "This is the third time I've had this issue, I want a refund now!"                                                                                                                                                                                         |
| 2    | Assistant (Complaint + Billing) | Acknowledges frustration, references the refund policy timeline from RefundPolicy.pdf, and appends an escalation notice: this case has been flagged for a human specialist who will follow up within 24 hours. escalated=true is returned to the frontend. |

## 22. Development Methodology and Timeline

The project was built incrementally, starting from the core backend pipeline and adding the frontend and deployment layers once the underlying agent logic was verified to work correctly against the knowledge base.

| Phase   | Focus                                                                                                               |
|---------|---------------------------------------------------------------------------------------------------------------------|
| Phase 1 | Backend skeleton: FastAPI app, health check, project structure (agents/, embeddings/, vectorstore/, rag/, models/)  |
| Phase 2 | RAG pipeline: PDF chunking, embedding generation, FAISS indexing, and the Retriever class                           |
| Phase 3 | Base agent and the five specialized agents, plus the Agent Router and intent classification (with keyword fallback) |
| Phase 4 | Authentication: password hashing, JWT issuing/verification, protected route dependency                              |
| Phase 5 | MongoDB integration: user storage, conversation history, feedback, and analytics aggregation                        |
| Phase 6 | Frontend: login/registration pages, chat interface, analytics dashboard, Axios service layer                        |
| Phase 7 | Manual testing across single-agent, multi-agent, and escalation scenarios                                           |
| Phase 8 | Deployment: frontend to Vercel, backend to Railway, database to MongoDB Atlas                                       |

## 23. Risk Analysis

| Risk                                                                             | Likelihood | Impact | Mitigation                                                                                                  |
|----------------------------------------------------------------------------------|------------|--------|-------------------------------------------------------------------------------------------------------------|
| Third-party LLM API outage or rate limiting                                      | Medium     | High   | Keyword-based fallback classifier keeps basic routing functional                                            |
| Retrieval returning irrelevant chunks for an ambiguous query                     | Medium     | Medium | Agent prompts instruct the model to say so honestly rather than guess when documents don't cover a question |
| JWT secret leakage or weak secret in production                                  | Low        | High   | Documented requirement to set a strong random JWT_SECRET via environment variable before deployment         |
| Unbounded conversation history inflating LLM prompt size/cost                    | Low        | Medium | get_history() is called with a fixed limit (10 most recent turns)                                           |
| Knowledge base growing beyond what an in-memory FAISS index can hold efficiently | Low        | Medium | Documented as a future consideration for a managed vector database (Section 18)                             |

## 24. Glossary

| Term             | Definition                                                                                                                     |
|------------------|--------------------------------------------------------------------------------------------------------------------------------|
| LLM              | Large Language Model — a neural network trained on large text corpora to generate and understand natural language              |
| RAG              | Retrieval-Augmented Generation — grounding an LLM's response in text retrieved from an external knowledge source at query time |
| Embedding        | A numeric vector representation of text such that semantically similar text produces similar vectors                           |
| FAISS            | Facebook AI Similarity Search — a library for efficient similarity search over dense vectors                                   |
| JWT              | JSON Web Token — a compact, signed token format used here to represent an authenticated session                                |
| Intent detection | The step that classifies a customer query into one or more predefined support categories                                       |
| Agent Router     | The orchestration component that dispatches a classified query to the matching specialized agent(s)                            |
| Escalation       | Automatically flagging a conversation for human intervention based on detected customer frustration                            |
| Chunking         | Splitting a long document into smaller overlapping text segments suitable for embedding and retrieval                          |

## 25. References

- FastAPI documentation — <https://fastapi.tiangolo.com/>
- LangChain documentation — <https://python.langchain.com/>
- Groq API documentation — <https://console.groq.com/docs>
- FAISS: A library for efficient similarity search — Meta AI Research
- sentence-transformers documentation (all-MiniLM-L6-v2) — <https://www.sbert.net/>
- MongoDB Atlas documentation — <https://www.mongodb.com/docs/atlas/>
- PyJWT / python-jose documentation for JWT handling in Python
- React and Vite documentation — <https://react.dev/>, <https://vitejs.dev/>
- Tailwind CSS documentation — <https://tailwindcss.com/docs>

## Appendix A: requirements.txt

```
fastapi
uvicorn
langchain-groq
langchain-core
sentence-transformers
faiss-cpu
python-dotenv
pydantic[email]
numpy
pymongo
bcrypt
python-jose[cryptography]
pypdf
```

## Appendix B: Environment Variables (.env)

```
GROQ_API_KEY=<your Groq API key>  
MONGODB_URI=<your MongoDB Atlas connection string>  
MONGODB_DB_NAME=<database name>  
JWT_SECRET=<a long random string>
```

On the frontend, VITE\_API\_URL should point to the deployed backend base URL (e.g. <https://customer-support-ai-production-2f5f.up.railway.app>).

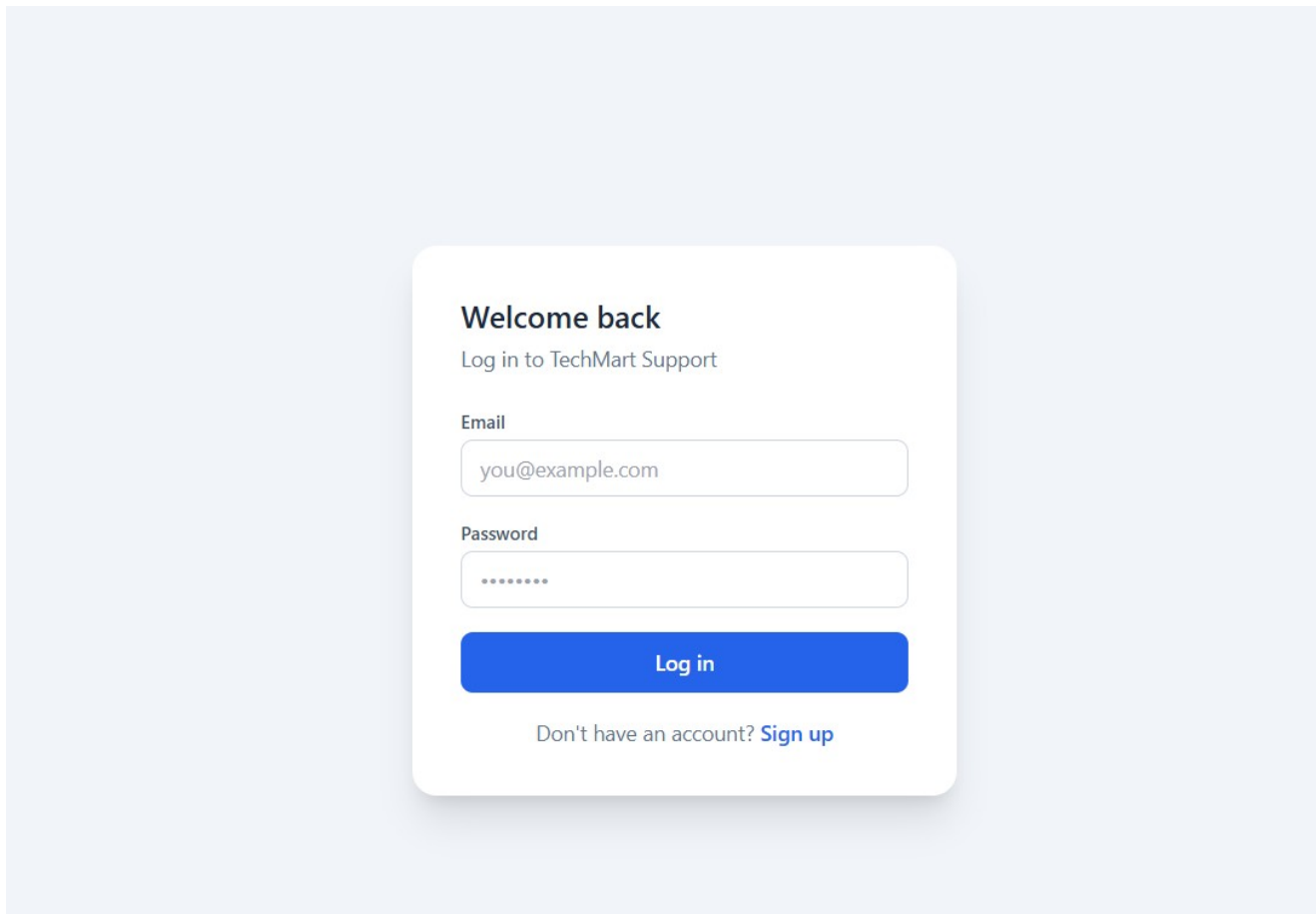


## Appendix C: Live Deployment Links

| Component         | URL                                                                                                                         |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Frontend (Vercel) | <a href="https://customer-support-qawhdibb8-anveiksha.vercel.app">customer-support-qawhdibb8-anveiksha.vercel.app</a>       |
| Backend (Railway) | <a href="https://customer-support-ai-production-2f5f.up.railway.app">customer-support-ai-production-2f5f.up.railway.app</a> |

## Appendix D: Screenshots

**Figure 1: Login Page**



**Figure 2: Sign-Up Page**

**Create your account**  
Sign up for TechMart Support

**Name**

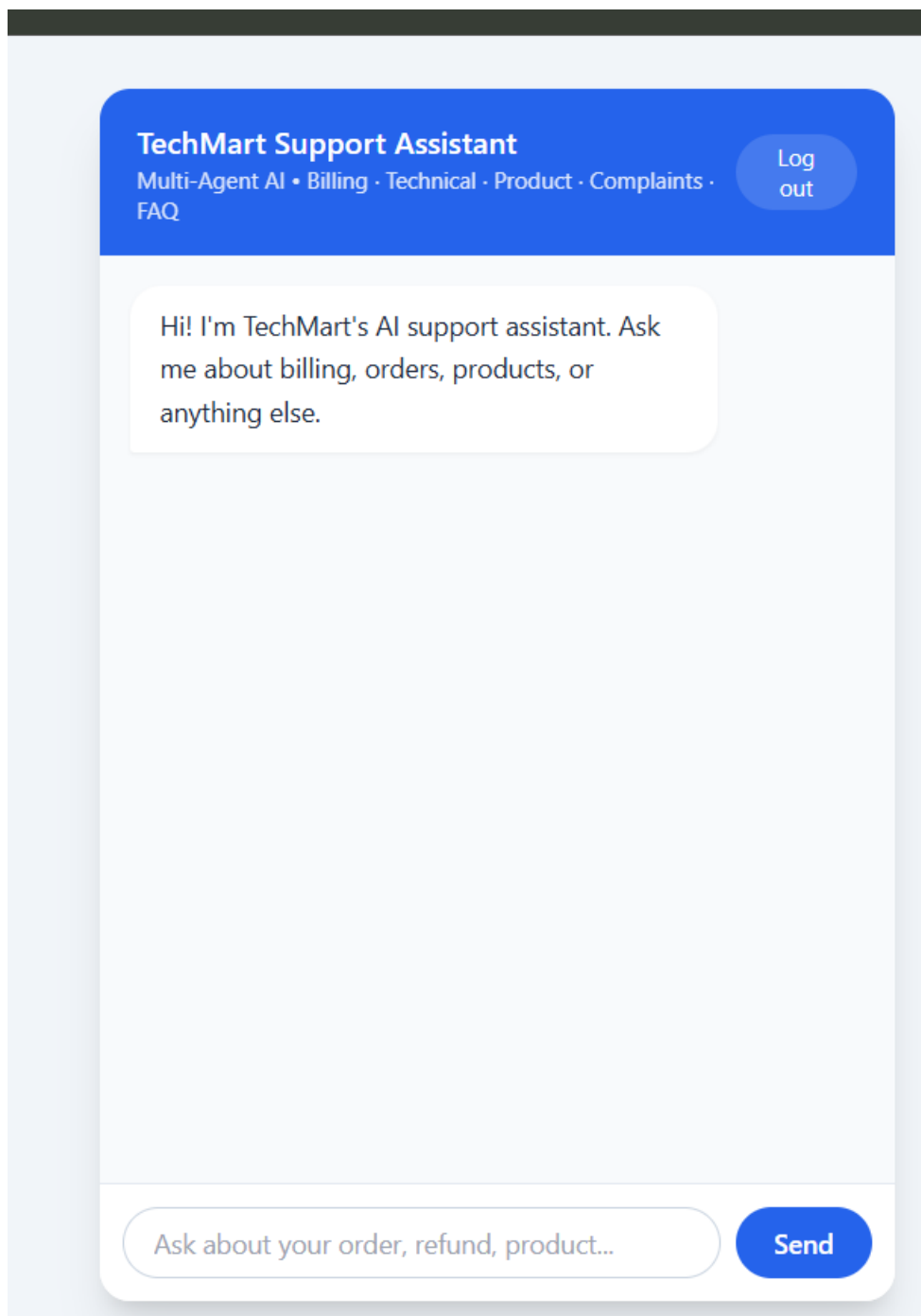
**Email**

**Password**

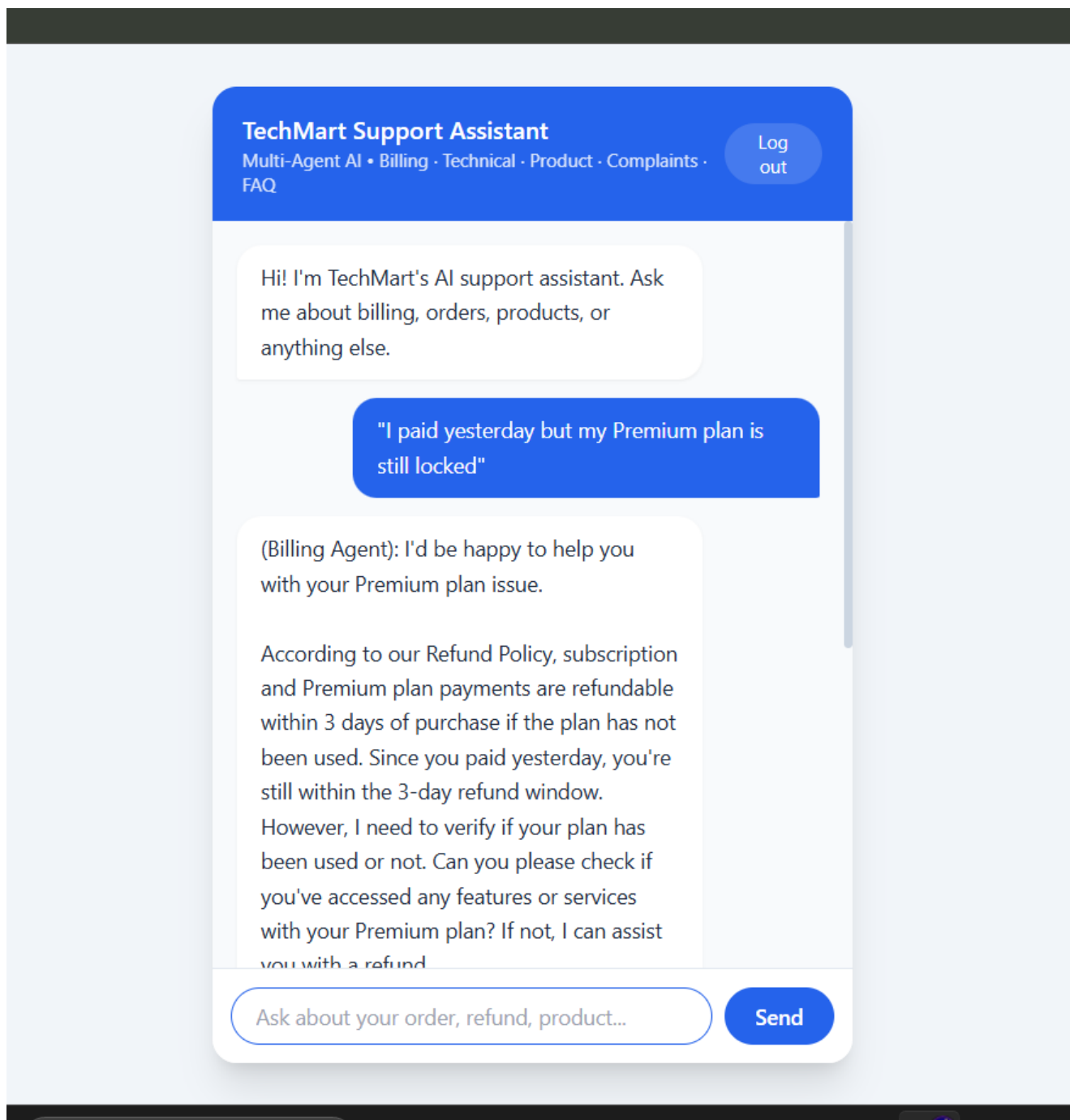
**Sign up**

Already have an account? [Log in](#)

**Figure 3: Chat Interface -- Single-Agent Response**



**Figure 4: Chat Interface -- Multi-Agent Response**



**Figures 5-7: Complaint Escalation Flow**

**TechMart Support Assistant**

Multi-Agent AI • Billing • Technical • Product • Complaints • FAQ

Log  
out

still within the 3-day refund window.  
However, I need to verify if your plan has been used or not. Can you please check if you've accessed any features or services with your Premium plan? If not, I can assist you with a refund.

(Technical Support Agent): Hello, I'm happy to help you with your Premium plan issue.

According to our Refund Policy, subscription and Premium plan payments are refundable within 3 days of purchase if the plan has not been used. Since you paid yesterday, you're still within the refund window. However, I need to confirm if you've used any features of the Premium plan. Can you please check if you've accessed any Premium features or services?

Handled by: Billing Agent, Technical Support Agent

Send

**TechMart Support Assistant**

Multi-Agent AI • Billing • Technical • Product • Complaints • FAQ

Log  
out

of the Premium plan. Can you please check if you've accessed any Premium features or services?

Handled by: Billing Agent, Technical Support Agent

This is the third time I've had this issue, I want a refund now!

(Billing Agent): I understand your frustration, and I'm here to help. However, according to our Refund Policy, refunds are only available within 3 days of purchase if the Premium plan has not been used. Since you paid yesterday, you're still within the 3-day window. I'd be happy to assist you with a refund if you haven't used any features or services with your Premium plan.

If you're still experiencing issues, I can offer you an option to escalate this to a human agent who can further assist you. Would

Ask about your order, refund, product...

Send

**TechMart Support Assistant**

Multi-Agent AI • Billing • Technical • Product • Complaints • FAQ

Log  
out

(Complaint Agent): I understand your frustration, and I apologize for the inconvenience you've experienced with your Premium plan. However, according to our Refund Policy, a refund is only available within 3 days of purchase if the plan has not been used. Since you paid yesterday, you're still within the refund window. I'd be happy to assist you with a refund if you haven't used any Premium features or services. Please check if you've accessed any Premium features or services. If you have, I'll need to explore other options with you.

This case has been flagged for escalation to a human support specialist, who will follow up within 24 hours.

**Handled by: Billing Agent, Complaint Agent •**  
**ESCALATED to human agent**

Ask about your order, refund, product...

Send



## Appendix E: Additional Code Listings

### E.1 backend/auth.py

```
JWT_SECRET = os.getenv("JWT_SECRET", "dev-only-change-this-secret")
JWT_ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24 # 24 hours

def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, JWT_SECRET, algorithm=JWT_ALGORITHM)

def get_current_user(token: str = Depends(oauth2_scheme)) -> dict:
    payload = decode_access_token(token)
    email = payload.get("sub")
    if not email:
        raise HTTPException(status_code=401, detail="Invalid token payload")
    return {"email": email}
```

### E.2 backend/api/auth\_routes.py

```
router = APIRouter(prefix="/auth", tags=["auth"])

@router.post("/register", response_model=TokenResponse)
def register(user: UserRegister):
    if database.get_user_by_email(user.email):
        raise HTTPException(status_code=400, detail="Email already registered")
    hashed = hash_password(user.password)
    database.create_user(user.name, user.email, hashed)
    token = create_access_token({"sub": user.email})
    return TokenResponse(access_token=token, name=user.name, email=user.email)

@router.post("/login", response_model=TokenResponse)
def login(credentials: UserLogin):
    user = database.get_user_by_email(credentials.email)
    if not user or not verify_password(credentials.password, user["password"]):
        raise HTTPException(status_code=401, detail="Invalid email or password")
    token = create_access_token({"sub": user["email"]})
    return TokenResponse(access_token=token, name=user["name"], email=user["email"])
```

### E.3 backend/api/analytics\_routes.py

```
router = APIRouter(tags=["analytics"])

@router.post("/feedback")
def submit_feedback(req: FeedbackRequest, current_user: dict = Depends(get_current_user)):
    database.save_feedback(req.session_id, req.agent, req.rating)
    return {"status": "recorded"}

@router.get("/analytics")
def analytics(current_user: dict = Depends(get_current_user)):
    return database.get_analytics()
```

### E.4 backend/llm.py

```

MODEL = "llama-3.1-8b-instant"
_llm = ChatGroq(model=MODEL, api_key=_api_key, temperature=0.3) if _api_key else None

_prompt_template = ChatPromptTemplate.from_messages([
    ("system", "{system_prompt}"),
    ("human", "{user_message}"),
])

def chat(system_prompt: str, user_message: str, temperature: float = 0.3) -> str:
    if _llm is None:
        return ("[LLM not configured] Please add your GROQ_API_KEY to the .env file. "
            "Get a free key at https://console.groq.com/keys")
    try:
        llm_with_temp = _llm.bind(temperature=temperature)
        chain = _prompt_template | llm_with_temp
        response = chain.invoke({"system_prompt": system_prompt, "user_message":
user_message})
        return response.content.strip()
    except Exception as e:
        return f"[LLM error] {e}"

```

## E.5 backend/models/user.py and feedback.py

```

class UserRegister(BaseModel):
    name: str
    email: EmailStr
    password: str

class UserLogin(BaseModel):
    email: EmailStr
    password: str

class TokenResponse(BaseModel):
    access_token: str
    token_type: str = "bearer"
    name: str
    email: str

class FeedbackRequest(BaseModel):
    session_id: str
    agent: str
    rating: str # "up" or "down"

```